# Cryptography

## Elliptic-Curve Cryptography

Ksawery Możdżyński

# Agenda For Today

**1. Shorter keys, same safety**
Why another kind of crypto

**2. A curve of points**
$y^2 = x^3 + ax + b \bmod p$

**3. Adding points**
Chord, tangent, formulas

**4. Multiplying by a number**
$kG$ and the ECDLP

**5. Agreeing on a key**
ECDH, X25519, HKDF

**6. Encrypting and signing**
ElGamal and ECDSA on a curve

**7. ECC in the real world**
Sizes, speed, where it lives

# Today you will need




version 3.8 or newer




any one you like




Task 1 and small sums

Nothing to install. Starter files for Task 2 to Task 6: **GitHub**

# Recall: Diffie-Hellman and RSA

**Diffie-Hellman**

both compute $g^{ab} \bmod p$

safe while the **discrete logarithm** is hard: from $g^a \bmod p$, find $a$

**RSA**

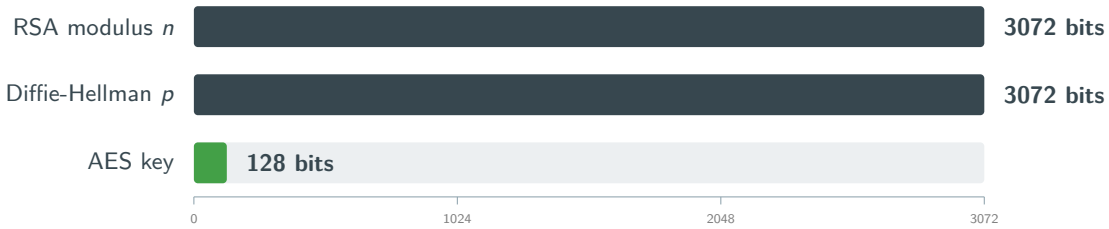
$C = P^e \bmod n$

safe while **factoring** is hard: split $n$ into its primes

Both use a **public key** that everybody may know and a **private key** that only its owner knows.

# The price: long keys

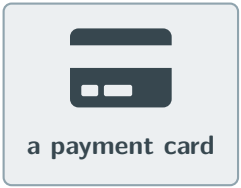
To be as hard to break as trying $2^{128}$ keys:



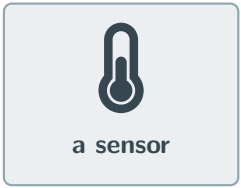

The better the attacks, the longer the keys must be.

Fast attacks on factoring and discrete logarithms force very long keys.

# Where long keys hurt




a small chip,
little memory, little power




a battery that
must last years



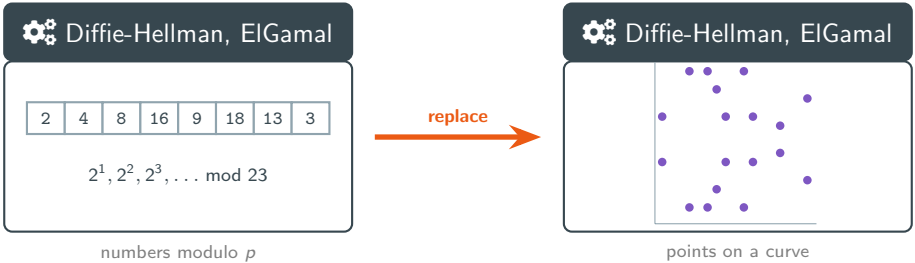

thousands of
connections a day




sent with every
HTTPS connection

# The idea: a harder group

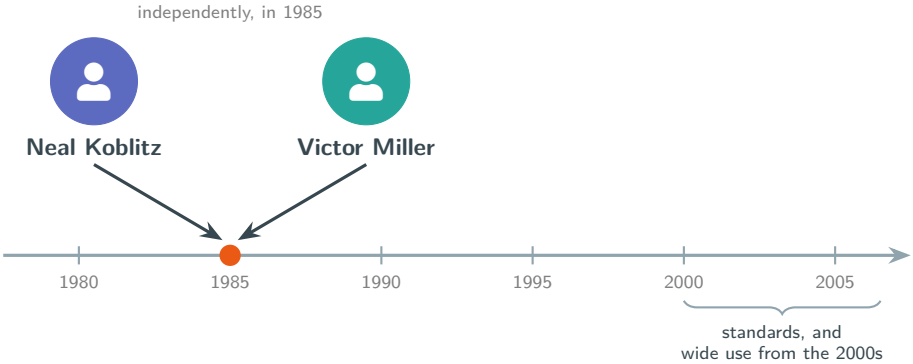
Keep Diffie-Hellman and ElGamal exactly as they are, but replace "numbers modulo $p$" with "points on a curve".




On a good curve, the best known attack is plain guessing. So **256 bits** give **128 bits** of security.

The same ideas on a curve: keys about 12 times shorter.

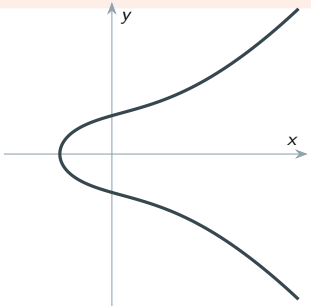
# Elliptic-curve cryptography, 1985




**ECC**: elliptic-curve cryptography.

# An elliptic curve

**Elliptic curve**: all points $(x, y)$ with $y^2 = x^3 + a\,x + b$, for chosen numbers $a$ and $b$.




Here: $a = 2$, $b = 2$:

$$y^2 = x^3 + 2x + 2$$

over the real numbers.

One line, the same above and below the $x$-axis.

# Real numbers will not do

Computers need exact numbers. Over the reals, points have endless decimals:

$$x = 1, \quad y = \sqrt{5} = 2.236067977499789696409173668731276235\cdots$$

**never ends, never exact**

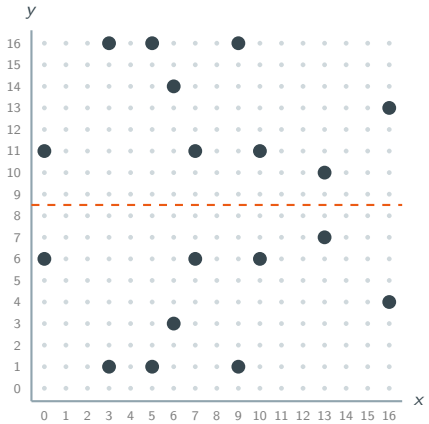
So we do everything **modulo a prime** $p$, as in Diffie-Hellman.

# The same curve modulo 17

$$y^2 = x^3 + 2x + 2 \bmod 17$$

$x$ and $y$: whole numbers from 0 to 16.

- a point of the curve
- not on the curve
- the line $y = 8.5$: $(x, y)$ and $(x, 17 - y)$ are mirror images




Modulo $p$, the curve is a cloud of dots.

# Is (5, 1) on the curve?

$x = 5,\ y = 1$

**left side**

$$y^2 = 1^2 = 1$$

**right side**

$$x^3 + 2x + 2 = 125 + 10 + 2 = 137$$

$$137 = 8 \times 17 + 1 \ \rightarrow\ \mathbf{1} \bmod 17$$



Both sides are 1: $(5, 1)$ is on the curve.

# All the points

18 points with $x$ and $y$ from 0 to 16, in mirror pairs $(x, y)$ and $(x, 17 - y)$:

(0,6) (3,1) (5,1) (6,3) (7,6) (9,1) (10,6) (13,7) (16,4)

(0,11) (3,16) (5,16) (6,14) (7,11) (9,16) (10,11) (13,10) (16,13)

+ O

the point at infinity

Plus one extra point: the **point at infinity** $O$.

This curve has 19 points: 18 dots and $O$.

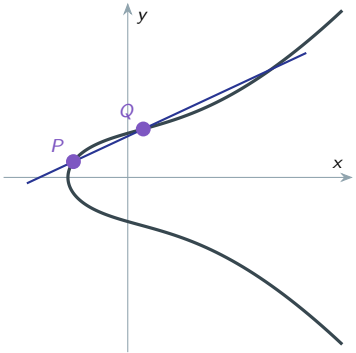
# Real curves

| | curve | prime $p$ | number of points |
|---|---|---|---|
|  | our toy curve | 17 | 19 |
|  | P-256 (NIST) | $2^{256} - 2^{224} + \ldots$ | about $2^{256}$ |
|  | Curve25519 | $2^{255} - 19$ | about $2^{252}$ in its main group |

Too many points to list, but the rules are the same.

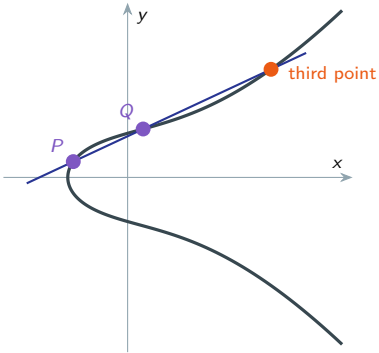
# Adding two points: the idea

1. Draw a straight line through $P$ and $Q$.



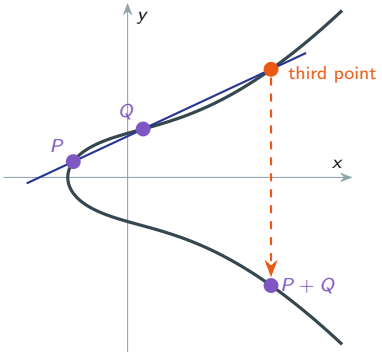
# Adding two points: the idea

1. Draw a straight line through $P$ and $Q$.
2. It meets the curve in exactly **one more point**.



# Adding two points: the idea

1. Draw a straight line through $P$ and $Q$.
2. It meets the curve in exactly **one more point**.
3. Reflect that point in the $x$-axis: this is $\mathbf{P + Q}$.

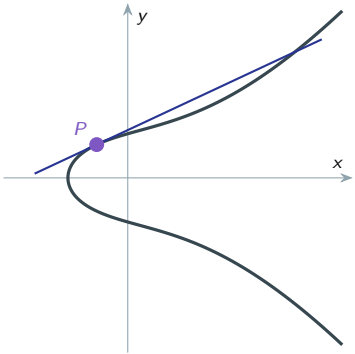



$P + Q$: draw the line, find the third point, reflect it.

# Doubling a point

To add $P$ to itself, the line through $P$ and $P$ is the **tangent** at $P$.

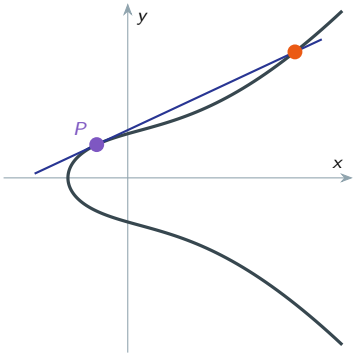
1. Draw the tangent at $P$.



# Doubling a point

To add $P$ to itself, the line through $P$ and $P$ is the **tangent** at $P$.

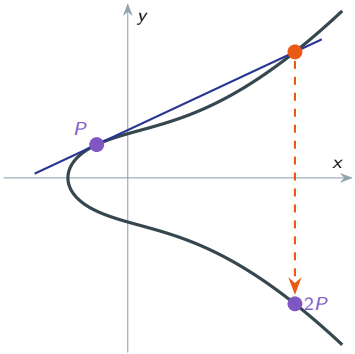
1. Draw the tangent at $P$.
2. Find where it meets the curve again.



# Doubling a point

To add $P$ to itself, the line through $P$ and $P$ is the **tangent** at $P$.

1. Draw the tangent at $P$.
2. Find where it meets the curve again.
3. Reflect: this is $P + P = \mathbf{2P}$.



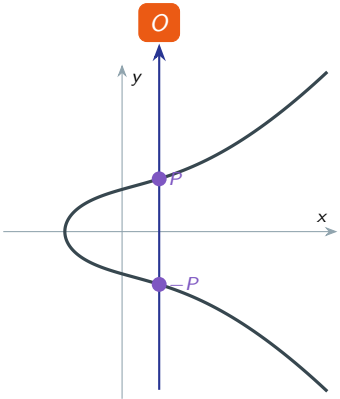
# P + (-P) = O

$-P$ is the reflection of $P$:

$$P = (x, y), \qquad -P = (x, -y)$$

The line through $P$ and $-P$ is **vertical**.
It never meets the curve again.

We say it meets the curve at $O$,
the **point at infinity**.




$O$ works like zero: $P + O = P$ and $P + (-P) = O$.

# The formulas

**slope** $s$: for $P + Q$: $s = (y_2 - y_1) / (x_2 - x_1) \mod p$

for $P + P$: $s = (3x_1^2 + a) / (2y_1) \mod p$

**result**: $x_3 = \mathbf{s}^2 - x_1 - x_2 \mod p$

$y_3 = \mathbf{s}(x_1 - x_3) - y_1 \mod p$

"$/\, v \bmod p$" means multiplying by the **modular inverse** of $v$: the number $v^{-1}$ with $v \cdot v^{-1} = 1 \bmod p$.

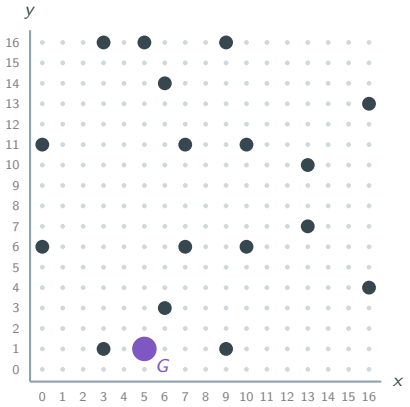
# Doubling G, step 1

$G = (5, 1)$ on $y^2 = x^3 + 2x + 2 \bmod 17$, $\quad 2G = ?$

- **the slope** $s = (3 \times 5^2 + 2)/(2 \times 1) = 77/2 \bmod 17$

  $77 \bmod 17 = 9, \quad 2^{-1} \bmod 17 = 9$

  $s = 9 \times 9 = 81 \bmod 17 = \mathbf{13}$



# Doubling G, step 2

$G = (5, 1)$ on $y^2 = x^3 + 2x + 2 \bmod 17$, $\quad 2G = ?$

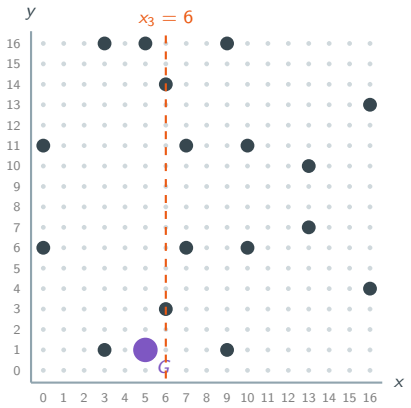
**the slope** $s = (3 \times 5^2 + 2)/(2 \times 1) = 77/2 \bmod 17$

$77 \bmod 17 = 9, \quad 2^{-1} \bmod 17 = 9$

$s = 9 \times 9 = 81 \bmod 17 = \mathbf{13}$

- **$x_3$** $x_3 = 13^2 - 5 - 5 = 159 \bmod 17$

  $x_3 = \mathbf{6}$



# Doubling G, step 3

$G = (5, 1)$ on $y^2 = x^3 + 2x + 2 \bmod 17$, $\quad 2G = ?$

**the slope** $s = (3 \times 5^2 + 2)/(2 \times 1) = 77/2 \bmod 17$

$77 \bmod 17 = 9, \quad 2^{-1} \bmod 17 = 9$

$s = 9 \times 9 = 81 \bmod 17 = \mathbf{13}$

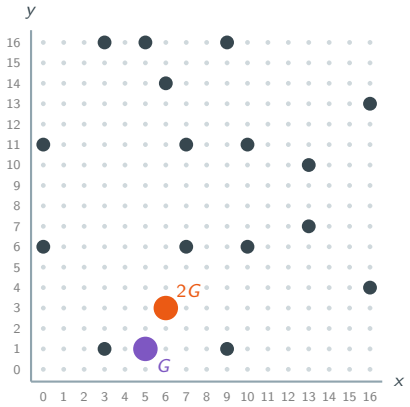
**$x_3$** $x_3 = 13^2 - 5 - 5 = 159 \bmod 17$

$x_3 = \mathbf{6}$

**$y_3$** $y_3 = 13 \times (5 - 6) - 1 = -14 \bmod 17$

$y_3 = \mathbf{3}$

**$2G = (6, 3)$**. Check: $3^2 = 9$ and $6^3 + 12 + 2 = 230 = 13 \times 17 + 9$.



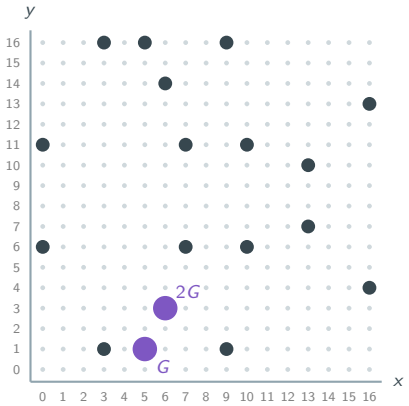

$2G = (6, 3)$ is again a point of the curve.

# Task 1: G + 2G on paper

**Task 1** (on paper)
$G = (5, 1), \quad 2G = (6, 3), \quad$ on $y^2 = x^3 + 2x + 2 \bmod 17$

1. Compute the slope $s$ for **two different points**.
2. Compute $x_3$ and $y_3$.
3. Check that the result is on the curve.

$$3G = G + 2G = (\ ?\ ,\ ?\ )$$



# Solution: G + 2G on paper

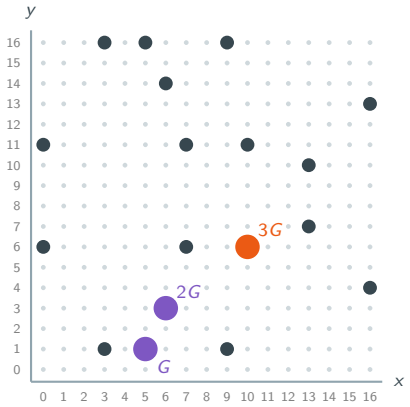
$s = (3 - 1)/(6 - 5) = 2/1 = \mathbf{2}$

$x_3 = 2^2 - 5 - 6 = -7 \bmod 17 = \mathbf{10}$

$y_3 = 2 \times (5 - 10) - 1 = -11 \bmod 17 = \mathbf{6}$

$\mathbf{3G} = (\mathbf{10}, \mathbf{6})$.

Check: $6^2 = 36 = 2 \bmod 17$, $1000 + 20 + 2 = 1022 = 2 \bmod 17$.

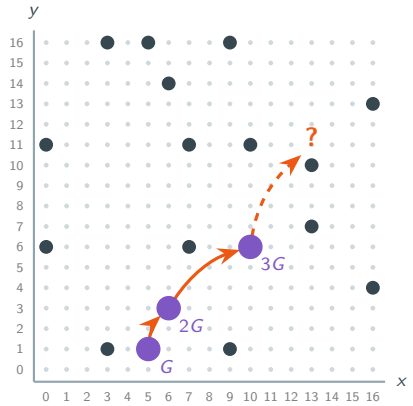


# Task 2: all points from G

**Task 2**
Write the point addition in Python, then add $G$ again and again.

$$G,\ 2G,\ 3G,\ 4G,\ \ldots$$

**Predict first**
How many different points will you get before you reach $O$?



# Task 2: what to do

Task2/ec_toy.py

```
def add(P, Q):
    if P is 0:
        return Q
    if Q is 0:
        return P
    (x1, y1), (x2, y2) = P, Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return 0
    # TODO 1: the slope s
    #   if P == Q (doubling):   s = (3 * x1^2 + a) / (2 * y1)   mod p
    #   otherwise:              s = (y2 - y1) / (x2 - x1)       mod p
    #   "/ v mod p" means "* pow(v, -1, p)"
    # TODO 2: x3 = s^2 - x1 - x2 mod p,   y3 = s * (x1 - x3) - y1 mod p
    return 0
# TODO 3: print G, 2G, 3G, ... by adding G again and again,
#         until you reach 0. How many points do you get?
```

**Task 2**

Open `ec_toy.py` from **GitHub**. Write the code in place of the `TODO` lines.

**Ready:** $O$ (`None`), and the cases $P + O$ and $P + (-P)$.

**You write:** the slope, $x_3$, $y_3$, and the loop.

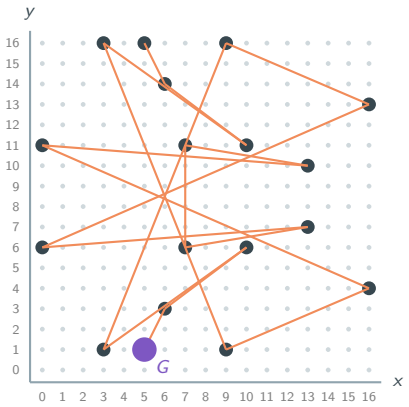
Inverse: `pow(v, -1, p)`

# Solution: all points from G

Task2/solution/ec_toy.py

```
if P == Q:
    s = (3 * x1 * x1 + a) * pow(2 * y1, -1, p) % p
else:
    s = (y2 - y1) * pow(x2 - x1, -1, p) % p
x3 = (s * s - x1 - x2) % p
y3 = (s * (x1 - x3) - y1) % p
```

```
G=(5,1)    2G=(6,3)    3G=(10,6)   4G=(3,1)
5G=(9,16)  6G=(16,13)  7G=(0,6)    ...
...        17G=(6,14)  18G=(5,16)  19G=O
```




*G* visits every point of the curve, then returns to *O* after 19 steps.

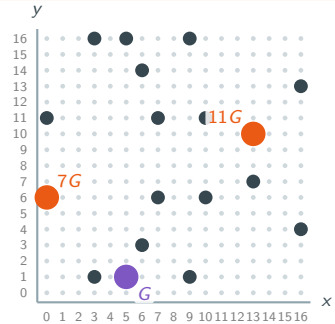
# Scalar multiplication

$\mathbf{k}\,\mathbf{G} = \mathbf{G} + \mathbf{G} + \ldots + \mathbf{G}$ ($k$ times). $k$ is a **number**, $G$ and $kG$ are **points**.

$7 \times G = (0, 6)$

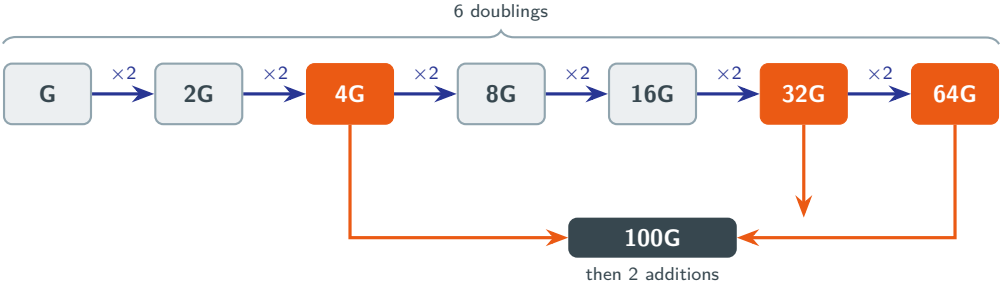
$11 \times G = (13, 10)$

$19 \times G = 0$



# Fast: double and add

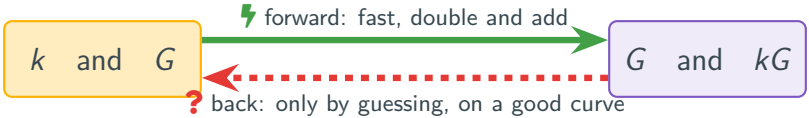
$$100\,G = 64G + 32G + 4G \qquad \text{because } 100 = 64 + 32 + 4$$




For a 256-bit $k$: about **256 doublings**, not $2^{256}$ additions.

# The hard problem: ECDLP

**Elliptic-curve discrete logarithm problem (ECDLP)**:
given $G$ and $T = k\,G$, find $k$.



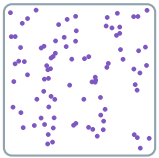

Multiplying a point is easy. Finding the multiplier is hard.

# Task 3: find k

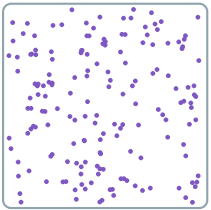
**Task 3**
Three curves, $p$ has 16, 20 and 24 bits. You know $G$ and $T = kG$. Find $k$ by adding $G$ again and again.



$p$: **16 bits**



$p$: **20 bits**



$p$: **24 bits**




**Predict first**
Each $p$ is 4 bits longer. How many times longer will it take?

# Task 3: what to do

Task3/ecdlp.py

```python
def find_k(p, a, G, T):
    # TODO: start with R = G and k = 1, keep adding G (R = add(R, G, a, p))
    #       until R == T, then return k
    return None
```

**Task 3**

Open `ecdlp.py` from **GitHub**. Write `find_k` in place of the `TODO` lines.

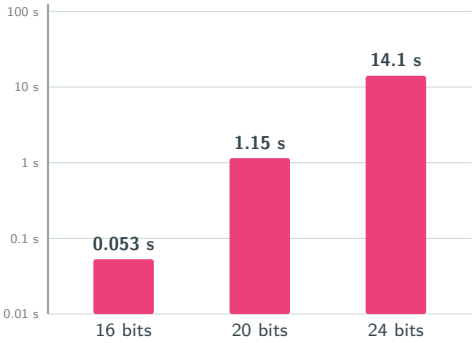
**Ready in the file**

`add(P, Q, a, p)`, the three curves, and the timing loop.

# Solution: find k

Task3/solution/ecdlp.py

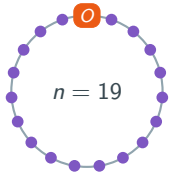
```python
def find_k(p, a, G, T):
    R, k = G, 1
    while R != T:
        R = add(R, G, a, p)
        k += 1
    return k
```



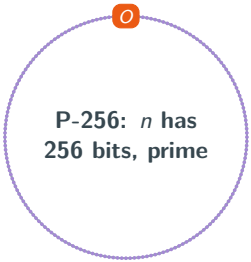

Every 4 more bits make the search more than 10 times longer.

# The order of G

The **order** of $G$: the smallest $n$ with $n\,G = O$.




our curve: only 19 multiples, try them all




about $2^{256}$ multiples

$G$ must have a huge prime order, or $kG$ takes too few values.

The choice of $G$ matters: a large prime order, or the curve is weak.

# Good to know: where do the constants come from?



**P-256** (NIST, 2000)

```
b = 5ac635d8 aa3a93e7 b3ebbd55 769886bc
    651d06b0 cc53b0f6 3bce3c3e 27d2604b
```

comes from the hash of a **seed nobody can explain**



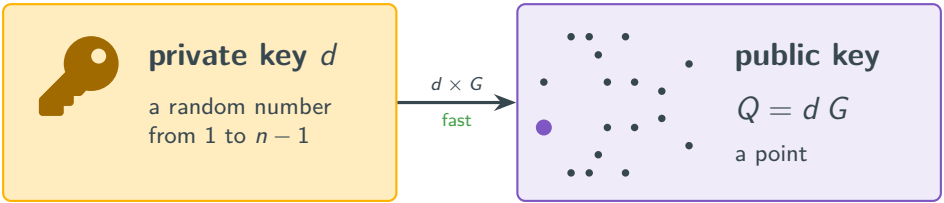
**Curve25519** (2006)

$y^2 = x^3 + \mathbf{486662}\,x^2 + x \bmod 2^{255} - 19$

every constant is the **smallest** that meets the stated rules

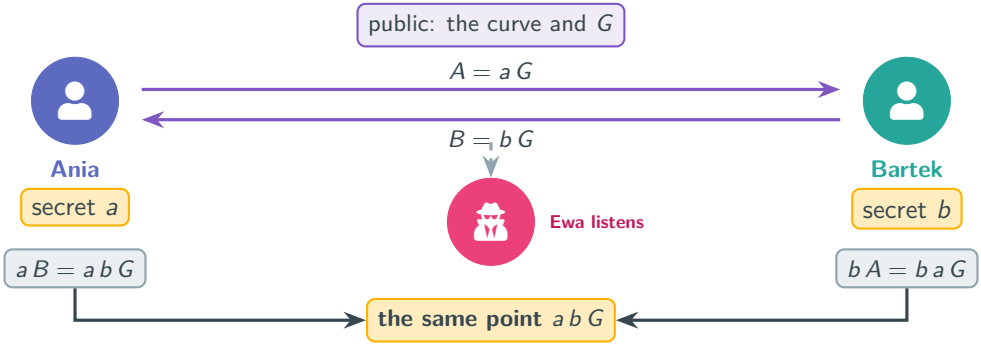
Many prefer curves whose constants anybody can check.

# Keys on a curve




The ECDLP protects $d$: from $Q$ and $G$, nobody can find it.

# ECDH




Both get the same point $a\,b\,G$. Ewa saw only $G$, $A$ and $B$.

ECDH is Diffie-Hellman with points instead of powers.

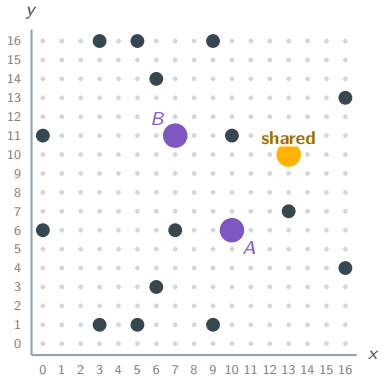
# ECDH on our curve

**Ania**: $a = 3$ $\quad A = 3G = \mathbf{(10,\ 6)}$

**Bartek**: $b = 10$ $\quad B = 10G = \mathbf{(7,\ 11)}$

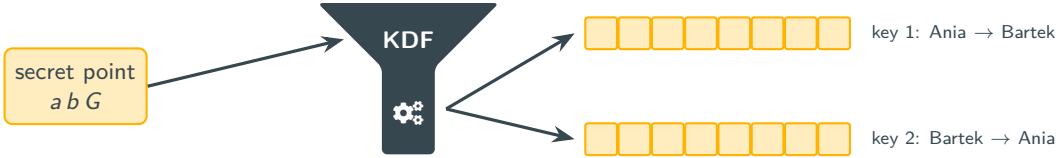
**Ania**: $\quad 3\,B = 3 \times (7, 11) = \mathbf{(13,\ 10)}$

**Bartek**: $\quad 10\,A = 10 \times (10, 6) = \mathbf{(13,\ 10)}$



# A point is not a key

The shared secret is a **point**. ChaCha20 or AES needs **32 random-looking bytes**.

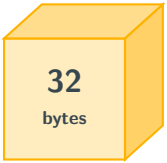



**KDF**: key derivation function. secret point → one or more symmetric keys.

The common choice is **HKDF** (RFC 5869), built on a hash function.

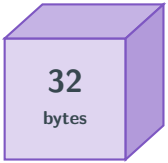
# X25519

ECDH on Curve25519, as used on the internet (RFC 7748):



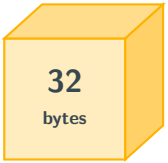

**private key**

32 random bytes




**public key**

sent to the other side




**shared key**

the same on both sides

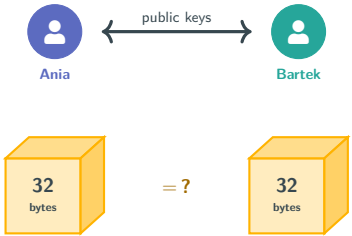
Designed to be hard to implement wrongly.

# Task 4: X25519

**Task 4**
A ready X25519 module in plain Python.

1. Compute Ania's and Bartek's public keys.
2. Compute the shared secret on both sides.
3. Derive a ChaCha20 key with HKDF.

The private keys are the test values from RFC 7748: compare your result.



# Task 4: what to do

Task4/ecdh.py

```python
from x25519_ready import public_key, x25519, hkdf

# Private keys: 32 random bytes each (here the test values from RFC 7748, section 6.1)
ania_private = bytes.fromhex("77076d0a7318a57d3c16c17251b26645df4c2f87ebc0992ab177fba51db92c2a")
bartek_private = bytes.fromhex("5dab087e624a8a4b79e17f8b83800ee66f3bb1292618b6fd1c2f8b27ff88e0eb")

# TODO 1: the public keys: public_key(private)            (this is d x G on Curve25519)
# TODO 2: the shared secret on both sides: x25519(my_private, other_public)
# TODO 3: print both secrets (.hex()) and whether they are equal
# TODO 4: derive a 32-byte key for ChaCha20: hkdf(secret, b"lecture 5 chat key")
#         and print it (.hex())
```

**Task 4**

Open `ecdh.py` from **GitHub**. Write the code in place of the `TODO` lines.

**Ready in `x25519_ready.py`**

`public_key(private)`, `x25519(private, public)`, `hkdf(secret, info)`

# Solution: X25519

Task4/solution/ecdh.py

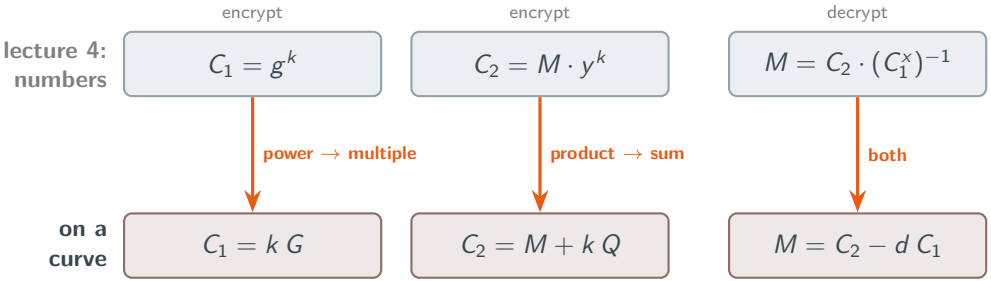
```
ania_public = public_key(ania_private)
bartek_public = public_key(bartek_private)
secret_ania = x25519(ania_private, bartek_public)
secret_bartek = x25519(bartek_private, ania_public)
print("equal:", secret_ania == secret_bartek)
key = hkdf(secret_ania, b"lecture 5 chat key")
```

```
secret:  4a5d9d5ba4ce2de1728e3bf480350f25e07e21c947d19e3376f09b3c1e161742
equal:   True
key:     8232af18b2d158a867dead920cb27b5f70409d17ef4712ba664eb4d2ff392f78
```

The same 32-byte secret on both sides, as in RFC 7748.

# ElGamal on a curve

**Bartek**: private $d$, public $Q = d\,G$




Encrypt a point $M$: a random $k$, $C_1 = k\,G$, $C_2 = M + k\,Q$. Decrypt: $M = C_2 - d\,C_1$.

Exactly ElGamal: powers became multiples, products became sums.

# The example on our curve

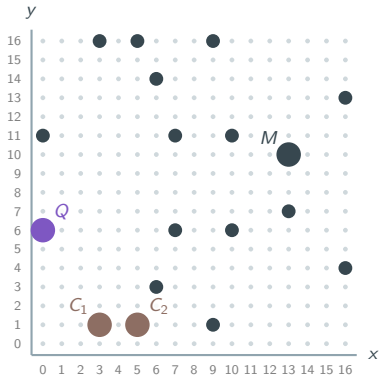
$d = 7, \quad Q = 7G = (0, 6), \quad M = (13, 10), \quad k = 4$

$C_1 \quad = 4G = \mathbf{(3,\ 1)}$

$C_2 \quad = M + 4Q = (13, 10) + (7, 6) = \mathbf{(5,\ 1)}$

**decrypt**: $d\,C_1 = 7 \times (3, 1) = (7, 6)$

$C_2 - (7, 6) = (\mathbf{13}, \mathbf{10})$ ✔



# Task 5: ElGamal on a curve

**Task 5**
The small curve, Bartek's keys $d = 7$ and $Q = 7G$, the message $M = (13, 10)$.

1. Write `encrypt(M, Q, k)`.
2. Write `decrypt(C1, C2, d)`.

```
def encrypt(M, Q, k):
    # TODO
```

```
def decrypt(C1, C2, d):
    # TODO
```

**Ready**
`add`, `mul` and `neg` (the point $-P$).

# Task 5: what to do

Task5/ec_elgamal.py

```python
def encrypt(M, Q, k):
    # TODO 1: C1 = k G,  C2 = M + k Q.  Return (C1, C2).
    return None


def decrypt(C1, C2, d):
    # TODO 2: S = d C1,  M = C2 - S  (that is: C2 + neg(S)).  Return M.
    return None
```

**Task 5**

Open `ec_elgamal.py` from **GitHub**. Write the code in place of the `TODO` lines.

**Ready in the file**

`add(P, Q)`, `mul(k, P)`, `neg(P)`, $G$, $d$, $Q$ and $M$.

# Solution: ElGamal on a curve

Task5/solution/ec_elgamal.py

```python
def encrypt(M, Q, k):
    return mul(k, G), add(M, mul(k, Q))


def decrypt(C1, C2, d):
    S = mul(d, C1)
    return add(C2, neg(S))
```

```
$ ciphertext (C1, C2):  ((3, 1), (5, 1))
```

```
$ decrypted:  (13, 10)
```

# ECDSA: signing with a curve

private $d$, public $Q = d\,G$, $n$ = the order of $G$

**sign** a message $m$:

a random $k$, $r$ = the $x$-coordinate of $\mathbf{k}\,G \bmod n$

$s = \mathbf{k}^{-1}\left(\mathrm{hash}(m) + r\,\mathbf{d}\right) \bmod n$ signature $(r, s)$



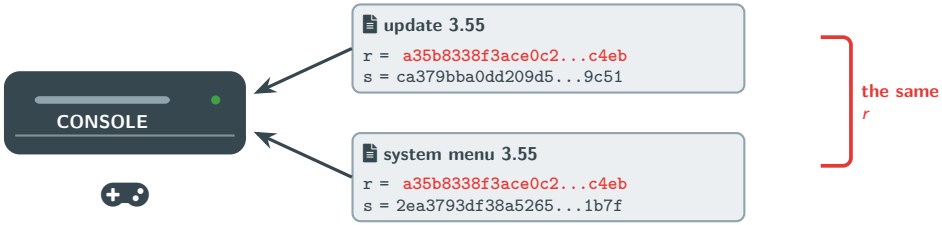
**verify**: with $Q$ only, checks that $r$ matches



**ECDSA**: the elliptic-curve digital signature algorithm.

# A famous case: the PS3

A games console runs only programs signed with ECDSA by its maker.




2010: researchers (fail0verflow) notice that every signature has the same $r$.

**The same $r$ means the same $k$. Every time.**

# Two signatures, one k

$$s_1 = \mathbf{k}^{-1} \left(h_1 + r\,\mathbf{d}\right)$$
$$s_2 = \mathbf{k}^{-1} \left(h_2 + r\,\mathbf{d}\right) \qquad (\mathrm{mod}\, n)$$

**subtract**: $s_1 - s_2 = k^{-1}\left(h_1 - h_2\right) \quad \rightarrow \quad \mathbf{k} = \left(h_1 - h_2\right) / \left(s_1 - s_2\right)$

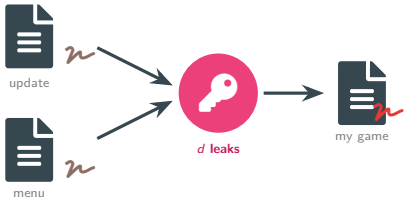
**then**: $\mathbf{d} = \left(s_1\, k - h_1\right) / r$

A repeated $k$ gives away the private key.

# Task 6: the same k twice

**Task 6**
The maker's public key and two signed updates with the same $r$, on the real curve P-256.

1. Compute $k$.
2. Compute the private key $d$ and check it against the public key.
3. Sign your own game so that the console accepts it.



# Task 6: what to do

Task6/reuse.py

```python
from p256 import G, n, mul, h, sign, verify
# Two signed updates. Look at r: it is the same in both, so the same k was used twice.
m1 = b'Game update 3.55, signed by the console maker'
m2 = b'System menu 3.55, signed by the console maker'
# s = k^-1 (h(m) + r d) mod n.  With the same k and r in both signatures:
# TODO 1: k = (h(m1) - h(m2)) * (inverse of (s1 - s2)) mod n
# TODO 2: d = (s1 * k - h(m1)) * (inverse of r1) mod n
# TODO 3: check that mul(d, G) == Q, then sign your own message with d:
#         sign(b"My homebrew game", d, some new k) and verify it with Q
```

**Task 6**

Open `reuse.py` from **GitHub**. Write the code in place of the `TODO` lines.

**Ready in `p256.py`**

`G`, `n`, `mul`, `h` (SHA-256 as a number), `sign`, `verify`.

Inverse mod $n$: `pow(x, -1, n)`

# Solution: the same k twice

Task6/solution/reuse.py

```
k = (h(m1) - h(m2)) * pow(s1 - s2, -1, n) % n
d = (s1 * k - h(m1)) * pow(r1, -1, n) % n
print("private key found:", mul(d, G) == Q)

mine = b"My homebrew game"
r, s = sign(mine, d, 123456789)
print("my game accepted:", verify(mine, r, s, Q))
```

```
$ private key found: True
```

```
$ my game accepted: True
```

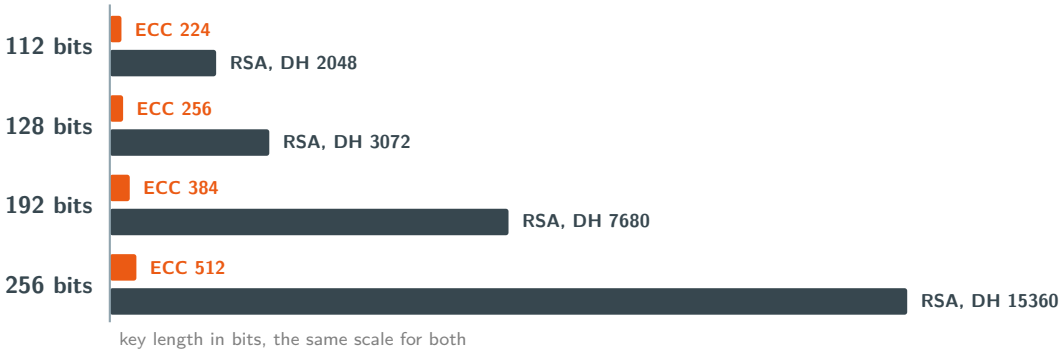
Two signatures with one $k$, and anybody can sign as the maker.

# ECC and ElGamal side by side

| | **ElGamal (numbers)** | **ElGamal on a curve** |
|---|---|---|
| operation | multiply mod $p$ | add points |
| repeated operation | power $g^k$ | multiple $k\,G$ |
| hard problem | discrete logarithm | ECDLP |
| key for 128 bits | 3072 bits | **256 bits** |
| trap | a repeated $k$ | a repeated $k$ |

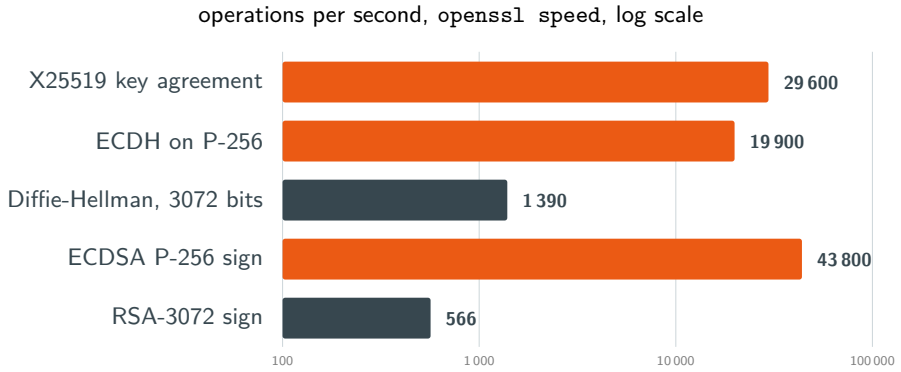
Same algorithm, a different group, keys 12 times shorter.

# Key sizes




ECC keys grow slowly. RSA keys grow very fast.

# Measured on a laptop




At the same security, ECC is 20 to 80 times faster at private-key work.

# Where ECC lives



**HTTPS**
TLS 1.3 key agreement, mostly X25519; ECDSA certificates



**phones**
short keys and fast operations save battery



**cryptocurrency**
Bitcoin signs transactions on the curve secp256k1



**messengers**
Signal: X25519 for keys, XEdDSA for signatures

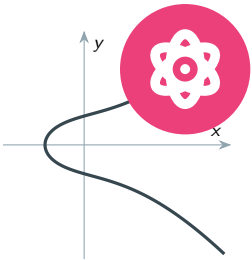


**smart cards**
small chips with little memory



**IoT**
sensors and small devices

# Good to know: quantum computers




**Shor's algorithm**
would solve the ECDLP too, on a big quantum computer.

**That is why**
X25519 is now often paired with a post-quantum algorithm.

# Summary

1. ECC gives the same tools as DH, ElGamal and DSA with keys about **12 times shorter**.
2. A curve mod $p$ is a set of points; we **add** them with lines and simple formulas.
3. $kG$ is fast by **double and add**; finding $k$ is the **ECDLP**; $G$ needs a large prime order.
4. **ECDH** (X25519) agrees on a point; a **KDF** such as HKDF turns it into a key.
5. ElGamal and **ECDSA** work on curves too; a **repeated** $k$ gives everything away.